

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №1
по дисциплине: **Операционные системы**
тема: **«Базовая работа с процессами в ОС Linux. Сигналы»**

Выполнил: ст. группы ВТ-231
Масленников Даниил

Проверили:
Островский Алексей Мичеславович

Белгород 2025 г

Лабораторная работа № 1

Цель работы: Изучить основы работы с процессами в ОС Linux, а также освоить сигналы как

универсальное средство межпроцессного взаимодействия.

Цель работы обуславливает постановку и решение следующих **задач**:

1. Изучить основные понятия о процессах в ОС Linux, относящиеся к их созданию и управлению.
2. Ознакомиться с базовыми системными вызовами POSIX для работы с процессами: `fork()`, `wait()`, `waitpid()`.
3. Изучить назначение и возможности сигналов в Linux, их классификацию (системные, пользовательские) и обработку.
4. Освоить установку обработчиков сигналов с помощью `signal()` и `sigaction()`, а также способы отправки сигналов с помощью `kill()` и `sigqueue()`.
5. Познакомиться с соответствующими консольными инструментами для мониторинга и управления процессами и сигналами.
 - 5.1 Изучение команд `ps`, `top`, `htop` — просмотр списка процессов, их состояний и ресурсов.
 - 5.2 Изучение команд `jobs`, `bg`, `fg` — управление фоновыми и приостановленными задачами в оболочке.
 - 5.3 Изучение команд `kill`, `pkill`, `killall` — отправка сигналов процессам по PID или имени.
 - 5.4 Изучение команды `ulimit` — просмотр и установка ресурсных ограничений (например, число процессов, размер стека).
 - 5.5 Изучение команды `strace` — отслеживание системных вызовов процесса, включая работу с сигналами.
6. Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).
7. Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Вариант 3

3. Породить кольцо из N процессов «родитель 0 → потомок 1 → потомок 2 → ... → потомок N-1 → родитель 0». Осуществить передачу по кольцу целого числа S, которое печатается в консоль текущим процессом вместе со своим PID и PPID сразу после получения S. Перед передачей S далее по кольцу значение S увеличивается текущим процессом на единицу. Для передачи S можно воспользоваться, например, библиотечной функцией стандарта POSIX `sigqueue`.

Выполнение индивидуального задания:

Исходный код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

#define NUM 5
#define SIG_NEXT SIGUSR2
#define SIG_DATA SIGUSR1

static volatile sig_atomic_t stop_flag = 0;
static pid_t next_pid = -1;
static int current_S = 0;

// Обработчик данных и настройки кольца
void data_handler(int sig, siginfo_t *info, void *ctx) {
    if (sig == SIG_NEXT) {
        next_pid = info->si_value.sival_int;
    } else if (sig == SIG_DATA) {
        current_S = info->si_value.sival_int;
        printf("[PID=%d, PPID=%d] S = %d\n", (int)getpid(), (int)getppid(), current_S);
        sleep(1);
        current_S++;
        if (next_pid != -1 && !stop_flag) {
            union sigval val = {.sival_int = current_S};
            sigqueue(next_pid, SIG_DATA, val);
        }
    }
}

// Обработчик завершения (для SIGINT и SIGTERM)
void stop_handler(int sig) {
    stop_flag = 1;
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[Main] Запуск кольца из %d процессов...\n", NUM);
```

```

// Устанавливаем обработчики
struct sigaction sa_data = {.sa_sigaction = data_handler, .sa_flags = SA_SIGINFO};
struct sigaction sa_stop = {.sa_handler = stop_handler};

sigaction(SIG_NEXT, &sa_data, NULL);
sigaction(SIG_DATA, &sa_data, NULL);
sigaction(SIGINT, &sa_stop, NULL);
sigaction(SIGTERM, &sa_stop, NULL);

pid_t children[NUM];
pid_t parent = getpid();

// Создаём дочерние процессы
for (int i = 0; i < NUM; i++) {
    if ((children[i] = fork()) == 0) {
        // Дочерний процесс: сбрасываем обработчики завершения на умолчание,
        // но оставляем обработчик данных
        signal(SIGINT, SIG_DFL);
        signal(SIGTERM, SIG_DFL);
        while (!stop_flag) {
            pause();
        }
        printf("[Child PID=%d] Завершаюсь.\n", (int) getpid());
        exit(0);
    }
}

// Настраиваем кольцо: parent → c0 → c1 → ... → c{NUM-1} → parent
for (int i = 0; i < NUM; i++) {
    pid_t next = (i == NUM - 1) ? parent : children[i + 1];
    union sigval val = {.sival_int = next};
    sigqueue(children[i], SIG_NEXT, val);
}
next_pid = children[0]; // родитель передаёт первому ребёнку

// Запускаем передачу
union sigval start = {.sival_int = 0};
sigqueue(children[0], SIG_DATA, start);

// Ждём сигнала завершения
while (!stop_flag) {
    pause();
}

```

```

}

printf("\n[Main] Получен сигнал завершения. Остановка кольца...\n");

// Посылаем SIGTERM всем детям
for (int i = 0; i < NUM; i++) {
    if (children[i] > 0) {
        kill(children[i], SIGTERM);
    }
}

// Ждём завершения всех
for (int i = 0; i < NUM; i++) {
    wait(NULL);
}

printf("[Main] Все процессы завершены. Выход.\n");
return 0;
}

```

Результат выполнения программы:

```

~/projects/university/5sem/oc/lab1 master* > ./res
[Main] Запуск кольца из 5 процессов...
[PID=22045, PPID=22044] S = 0
[PID=22046, PPID=22044] S = 1
[PID=22047, PPID=22044] S = 2
[PID=22048, PPID=22044] S = 3
[PID=22049, PPID=22044] S = 4
[PID=22044, PPID=17638] S = 5
[PID=22045, PPID=22044] S = 6
[PID=22046, PPID=22044] S = 7
[PID=22047, PPID=22044] S = 8
[PID=22048, PPID=22044] S = 9
[PID=22049, PPID=22044] S = 10
[PID=22044, PPID=17638] S = 11
[PID=22045, PPID=22044] S = 12

```

```
~/projects/university/5sem/oc/lab1 master* > ps aux | grep ./res
daancho+  22854  0.0  0.0  2572  1672 pts/0    S+   09:37   0:00 ./res
daancho+  22855  0.0  0.0  2572  1320 pts/0    S+   09:37   0:00 ./res
daancho+  22856  0.0  0.0  2572  1320 pts/0    S+   09:37   0:00 ./res
daancho+  22857  0.0  0.0  2572  1320 pts/0    S+   09:37   0:00 ./res
daancho+  22858  0.0  0.0  2572  1320 pts/0    S+   09:37   0:00 ./res
daancho+  22859  0.0  0.0  2572  1320 pts/0    S+   09:37   0:00 ./res
```

Вывод: В ходе выполнения лабораторной работы была успешно реализована система межпроцессного взаимодействия на основе сигналов POSIX. Программа создаёт кольцевую топологию из шести процессов (родительский и пять дочерних), по которой циклически передаётся целочисленное значение. Каждый процесс корректно обрабатывает входящий сигнал с помощью `sigaction()` и `sigqueue()`, выводит информацию о своём PID, PPID и полученном значении, увеличивает его на единицу и передаёт следующему участнику кольца.